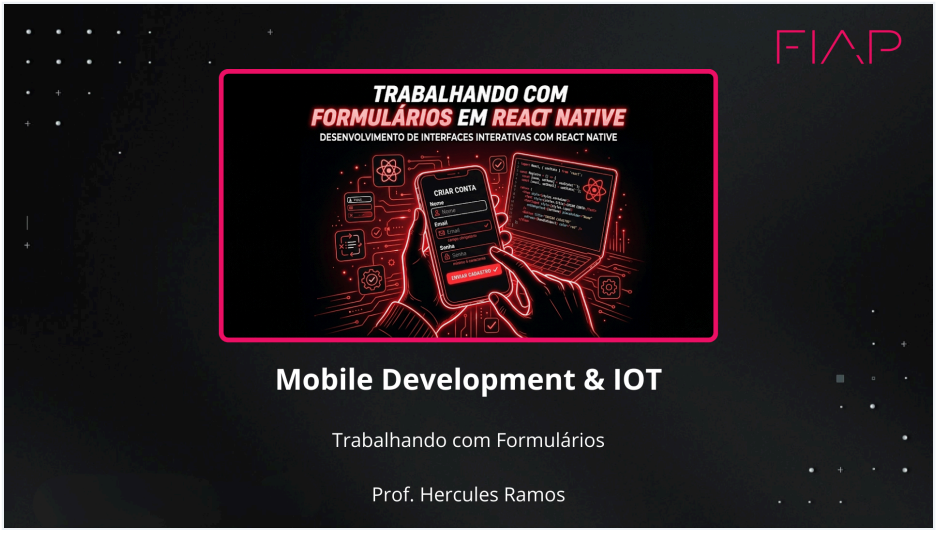
Aula MDI 08 - Trabalhando com Formulários



Engenharia de Software - 3º Ano

Mobile Development & IOT

A utilização do uso de soluções para mobile gera o aumento constante da demanda para o desenvolvimento de aplicações multiplaforma. Nesta matéria você será capacitado como um Mobile Developer, atendendo a crescente busca por profissionais com esse perfil. Isso significa ter o controle de múltiplas funções na palma da sua mão entendendo a fundo o conteúdo de desenvolvimento mobile & IoT, e integrando o melhor da sociedade 5.0 com os resultados obtidos da evolução da indústria 4.0.

> **"Um formulário mal feito é como uma fila do banco: todo mundo odeia, mas todo mundo tem que passar por ele."**

🗺️ O que vamos ver hoje?

1. Por que formulários são essenciais em apps
2. Componente `TextInput` — o coração dos formulários
3. Controlando estado com `useState`
4. Validação básica de dados
5. Teclado e UX — o app que não irrita o usuário
6. 🛠️ Projeto prático: **Mini Formulário de Cadastro**

1. Por que formulários existem?

Pensa em qualquer app que você usa hoje:

App	Formulário
📦 **iFood**	Endereço de entrega
💚 **Nubank**	Cadastro e login
🎵 **Spotify**	Criar conta
💬 **WhatsApp**	Verificação de número

> **Formulários são a porta de entrada do usuário para o sistema.**
> Se a porta está torta, o usuário não entra — ou entra xingando.

2. TextInput — O campo de formulário

O `TextInput` é o componente base para **capturar dados do usuário**.

```
import { TextInput, View } from 'react-native';
export default function App() {
  return (
    <View>
      <TextInput
        placeholder="Digite seu nome"
        style={{ borderWidth: 1, padding: 10, borderRadius: 8 }}
      />
    </View>
  );
}
```

Props essenciais do TextInput

Prop	O que faz
`placeholder`	Texto de dica quando vazio
`value`	Valor atual do campo
`onChangeText`	Função chamada ao digitar

Prop	O que faz
`secureTextEntry`	Esconde o texto (senha 🔒)
`keyboardType`	Tipo de teclado (`email-address`, `numeric`, `phone-pad`)
`autoCapitalize`	Capitalização automática (`none`, `sentences`, `words`)

---

3. Controlando o estado com useState

Sem estado, o campo não "lembra" o que o usuário digitou.

> 🧠 **Metáfora:** O `useState` é o **bloquinho de notas** do componente. Cada tecla digitada atualiza o bloquinho.

```
import { useState } from 'react';
import { TextInput, Text, View } from 'react-native';
export default function App() {
  const [nome, setNome] = useState('');
  return (
    <View style={{ padding: 20 }}>
      <TextInput
        placeholder="Qual é o seu nome?"
        value={nome}
        onChangeText={setNome}
        style={{ borderWidth: 1, padding: 10, borderRadius: 8 }}
      />
      <Text>Olá, {nome || '...'}! 👋</Text>
    </View>
  );
}
```

Múltiplos campos — sem mistério

```
const [email, setEmail] = useState('');
const [senha, setSenha] = useState('');
```

Cada campo tem **seu próprio estado**. Simples assim.

---

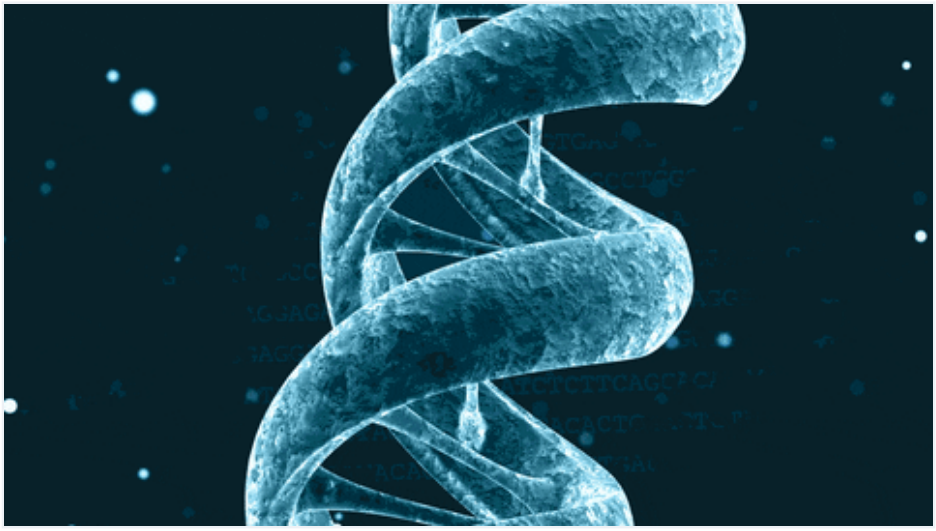
4. Capturando e exibindo dados — Formulário completo básico

```
import { useState } from 'react';
import { View, Text, TextInput, Button, Alert, StyleSheet } from 'react-native';
export default function App() {
  const [nome, setNome] = useState('');
  const [email, setEmail] = useState('');
  const handleSubmit = () => {
    Alert.alert('Dados enviados!', `Nome: ${nome}\nEmail: ${email}`);
  };
  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Cadastro</Text>
```

```
12        <TextInput
13          placeholder="Nome completo"
14          value={nome}
15          onChangeText={setNome}
16          style={styles.input}
17        />
18        <TextInput
19          placeholder="E-mail"
20          value={email}
21          onChangeText={setEmail}
22          keyboardType="email-address"
23          autoCapitalize="none"
24          style={styles.input}
25        />
26        <Button title="Cadastrar" onPress={handleSubmit} />
27      </View>
28    );
29  }
30  const styles = StyleSheet.create({
31    container: { flex: 1, padding: 24, justifyContent: 'center' },
32    titulo: { fontSize: 24, fontWeight: 'bold', marginBottom: 20 },
33    input: {
34      borderWidth: 1, borderColor: '#ccc',
35      borderRadius: 8, padding: 12, marginBottom: 16,
36    },
37  });
```

---

5. Validação — Não deixe lixo entrar no sistema



"Lixo entra, lixo sai." — Lei fundamental de todo sistema.

Validação **antes** de enviar os dados é obrigação, não opcional!

```
1  const handleSubmit = () => {
2    if (!nome.trim()) {
3      Alert.alert('Erro', 'O nome é obrigatório!');
4      return;
```

```
  }
  if (!email.includes('@')) {
    Alert.alert('Erro', 'E-mail inválido!');
    return;
  }
  Alert.alert('Sucesso', 'Cadastro realizado! 🎉');
};
```

Exibindo erros inline (UX melhor)

```
const [erroNome, setErroNome] = useState('');

const validar = () => {
  if (!nome.trim()) {
    setErroNome('Nome obrigatório');
  } else {
    setErroNome('');
  }
};
// No JSX:
{erroNome ? <Text style={{ color: 'red' }}>{erroNome}</Text> : null}
```

---

6. UX com Teclado — Detalhes que fazem diferença

Sabe quando você tenta digitar e o **teclado tampa tudo**? Isso é crime contra UX.

KeyboardAvoidingView — Seu melhor amigo

```
import { KeyboardAvoidingView, Platform } from 'react-native';

<KeyboardAvoidingView
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
  style={{ flex: 1 }}
>
  {/* Seu formulário aqui */}
</KeyboardAvoidingView>
```

ScrollView para formulários longos

```
import { ScrollView } from 'react-native';

<ScrollView keyboardShouldPersistTaps="handled">
  {/* Campos do formulário */}
</ScrollView>
```

Dicas rápidas de UX

```
// Campo de senha
<TextInput secureTextEntry={true} placeholder="Senha" />

// Campo numérico (ex: CPF)
<TextInput keyboardType="numeric" placeholder="CPF" />
```

```

// Campo de telefone
<TextInput keyboardType="phone-pad" placeholder="Telefone" />

// Ir para próximo campo com o teclado
<TextInput returnKeyType="next" onSubmitEditing={() => emailRef.current.focus()} />
```

---

7. Campo de senha com toggle 👁️

Funcionalidade clássica — esconder/mostrar senha:

```
const [senhaVisivel, setSenhaVisivel] = useState(false);

<View style={{ flexDirection: 'row', alignItems: 'center' }}>
  <TextInput
    placeholder="Senha"
    secureTextEntry={!senhaVisivel}
    value={senha}
    onChangeText={setSenha}
    style={[styles.input, { flex: 1 }]}
  />
  <Text onPress={() => setSenhaVisivel(!senhaVisivel)}>
    {senhaVisivel ? '🙈' : '👁️'}
  </Text>
</View>
```

---

8. Checklist de um bom formulário em RN

✅ Campos com `placeholder` descritivos

✅ `keyboardType` correto para cada campo

✅ `secureTextEntry` em campos de senha

✅ Validação antes de submeter

✅ Feedback visual de erro

✅ `KeyboardAvoidingView` para não tapar os campos

✅ Botão de submit com feedback (`Alert` ou navegação)

🛠️ Exercício Prático — Mini App de Login

Objetivo

Criar um **app de tela de login** com validação básica usando **VS Code + Expo CLI**.

Passo 1 — Criar o projeto

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy Unrestricted

npx create-expo-app mini-login -t

cd mini-login

npx expo start
```

Passo 2 — Substituir o conteúdo de `App.js`

```
import { useState } from 'react';
import {
  View, Text, TextInput, TouchableOpacity,
  Alert, StyleSheet, KeyboardAvoidingView, Platform
} from 'react-native';

export default function App() {
  const [email, setEmail] = useState('');
  const [senha, setSenha] = useState('');
  const [senhaVisivel, setSenhaVisivel] = useState(false);
  const [erros, setErros] = useState({});
  const validar = () => {
    const novosErros = {};
    if (!email.includes('@')) novosErros.email = 'E-mail inválido';
    if (senha.length < 6) novosErros.senha = 'Senha deve ter mínimo 6 caracteres';
    setErros(novosErros);
    return Object.keys(novosErros).length === 0;
  };
  const handleLogin = () => {
```

```
20      if (validar()) {
21        Alert.alert('Login realizado!', `Bem-vindo, ${email}! 🎉`);
22      }
23    };
24    return (
25      <KeyboardAvoidingView
26        style={styles.container}
27        behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
28      >
29        <Text style={styles.titulo}>🔐 Login</Text>
30        <TextInput
31          placeholder="E-mail"
32          value={email}
33          onChangeText={setEmail}
34          keyboardType="email-address"
35          autoCapitalize="none"
36          style={styles.input}
37        />
38        {erros.email && <Text style={styles.erro}>{erros.email}</Text>}
39        <View style={styles.senhaContainer}>
40          <TextInput
41            placeholder="Senha"
42            value={senha}
43            onChangeText={setSenha}
44            secureTextEntry={!senhaVisivel}
45            style={[styles.input, { flex: 1, marginBottom: 0 }]}
46          />
47          <Text
48            onPress={() => setSenhaVisivel(!senhaVisivel)}
49            style={styles.olho}
50          >
51            {senhaVisivel ? '🙈' : '👁️'}
52          </Text>
53        </View>
54        {erros.senha && <Text style={styles.erro}>{erros.senha}</Text>}
55        <TouchableOpacity style={styles.botao} onPress={handleLogin}>
56          <Text style={styles.botaoTexto}>Entrar</Text>
57        </TouchableOpacity>
58      </KeyboardAvoidingView>
59    );
60  }
61  const styles = StyleSheet.create({
62    container: {
63      flex: 1, justifyContent: 'center',
64      padding: 24, backgroundColor: '#f5f5f5',
65    },
66    titulo: {
67      fontSize: 32, fontWeight: 'bold',
68      textAlign: 'center', marginBottom: 32, color: '#333',
69    },
70    input: {
71      backgroundColor: '#fff', borderWidth: 1, borderColor: '#ddd',
72      borderRadius: 10, padding: 14, marginBottom: 8, fontSize: 16,
```

```
  },
  senhaContainer: {
    flexDirection: 'row', alignItems: 'center',
    backgroundColor: '#fff', borderWidth: 1, borderColor: '#ddd',
    borderRadius: 10, marginBottom: 8,
  },
  olho: { padding: 14, fontSize: 20 },
  erro: { color: 'red', marginBottom: 8, marginLeft: 4 },
  botao: {
    backgroundColor: '#6c47ff', borderRadius: 10,
    padding: 16, marginTop: 16, alignItems: 'center',
  },
  botaoTexto: { color: '#fff', fontSize: 18, fontWeight: 'bold' },
});
```

✅ O que foi praticado neste exercício?

Conceito	Aplicado em
`TextInput` + `useState`	Campos de e-mail e senha
`keyboardType`	Campo de e-mail
`secureTextEntry`	Campo de senha
Toggle senha	Botão 👁️
Validação inline	Mensagens de erro em vermelho
`KeyboardAvoidingView`	Teclado não tampa os campos
`TouchableOpacity`	Botão estilizado

🎯 Desafio:

- Adicionar campo de **"Confirmar senha"** e validar se os dois são iguais
- Adicionar um campo de **"Nome de usuário"** antes do e-mail
- Mudar a cor do botão para **verde** quando os dados estiverem válidos

---

📚 Resumo da Aula

Tema	Conceito-chave
`TextInput`	Componente para capturar entrada do usuário
`useState`	Controla o valor de cada campo
Validação	Sempre verificar antes de submeter
`KeyboardAvoidingView`	Evita que o teclado tampe os campos
`secureTextEntry`	Oculta texto em campos de senha

Tema	Conceito-chave
`keyboardType`	Define o teclado correto para cada campo

🧑‍🏫 Dúvidas?

Contato:

📧 profhercules.ramos@fiap.com.br

💼 LinkedIn

#KeepCoding #ReactNative #FIAP

Let's code! 💻🔥

"O único modo de aprender programação é programando." - Todo desenvolvedor, sempre

---

Copyright © 2026 Prof. Hercules Ramos

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor (autor).